

RestroPOS: Security & Data Compliance Audit

This document details the Zero-Trust security hardening, vulnerability remediations, and data compliance standards implemented across the RestroPOS SaaS architecture.

1. Vulnerability Remediations (Fixed Exploits)

During our security sweep, we identified and remediated two critical security vulnerabilities that would otherwise block enterprise SaaS compliance in US/EU markets.

■ Remediated: Table IDOR & URL Spoofing

- **The Exploit:** The original QR ordering route used the table's permanent database ID in the URL (e.g., `/order/cuid12345`). A malicious user sitting at Table 3 could easily guess or scrape other IDs to view active checkout screens, inject fake items into Table 10's cart, or close out their bill.
- **The Solution (Session UUID Algorithm):**

We replaced database IDs with a cryptographically secure **Session UUID Framework**:

1. Scanning the QR code accesses a session-based URL `/order/session/[session_uuid]` (generated using a random UUIDv4).
2. **120-Minute Expiry:** The database tracks the session's active window (`expiresAt`). Once the time expires or the guest pays, the session is invalidated immediately.
3. **Steward PIN Verification:** When the session starts, ordering is locked (`isActive = false`). The table steward provides a 4-digit PIN generated by the POS dashboard. Once entered, the session is activated, preventing remote order spoofing by off-site actors.

■ Remediated: Cross-Tenant SQL Data Bleeding

- **The Exploit:** In multitenant applications, a developer forgot to append `restaurantId` to a database query, potentially leaking customer orders, inventories, or staff logs from restaurant A to restaurant B.
- **The Solution (Extended Client Isolation):**

We implemented a **Prisma Query Extension** that intercepts every query at the application level:

- If a cashier or manager is logged in, the system automatically appends `where: { restaurantId }` to every query, including read, write, update, and delete actions.

- This is enforced globally, making it impossible for coding mistakes to result in cross-tenant data leaks.

2. Core Security & Compliance Structures

To satisfy strict data standards (GDPR, CCPA, PCI-DSS), we implemented database-level security policies and anonymization rules:

```
[Customer Client] ■■(Session Cookie)■■> [Next.js App (Prisma Extension)]
■
(Database connection pool)
■
▼
[Neon Postgres Database]
■■ Row-Level Security (RLS)
■■ Data Masking (Dev Branches)
```

1. Row-Level Security (RLS)

We enabled Postgres RLS policies directly inside our Neon PostgreSQL database:

- **Staff Isolation:** Staff members can only view records where `restaurant_id` matches their own authenticated tenant token.
- **Customer Isolation:** Customers placing QR orders can only query or mutate orders directly bound to their specific, active `session_uuid`.

2. Anonymized Data Masking (Neon Beta)

To comply with GDPR and CCPA, developer/staging database branches must not expose actual customer PII (Personally Identifiable Information). We configured **Neon's Data Masking** rules for development branches:

Table | Sensitive Field | Masking Rule

:--- | :--- | :---

Customer | phone | Replaced with *****

Customer | email | Replaced with anon@masked.dev

Customer | name | Replaced with Guest-{id[:5]}

StaffProfile | phone | Replaced with *****

Reservation | guestPhone | Replaced with *****

This prevents user data from being visible during developer testing or staging builds.

3. Session Security

- **Cookie Attributes:** Session tokens are transmitted as HTTP-only cookies (`__Secure-neonauth.session_token / neonauth.session_token`).
- **XSS Mitigation:** Because the cookies are HTTP-only, they cannot be read or stolen by malicious browser scripts (preventing Cross-Site Scripting session hijack attacks).
- **SameSite Config:** Set to `Strict` to prevent cross-site request forgery (CSRF) attempts.